

# M2S-05: Step 5 — Execute: Migrate Configuration and Agents

---

**Series:** M2S — Managed to SaaS Migration | **Notebook:** 5 of 9 | **Phase:** Upgrade | **Step:** Execute | **Created:** March 2026 | **Last Updated:** 09/18/2026

With your SaaS environment prepared, it is time to execute the migration. This step covers deploying configurations via the SaaS Upgrade Assistant in dependency-ordered waves, redirecting OneAgents from Managed to SaaS, and validating data flow after each wave. By the end of this step, all hosts and services will be reporting to your SaaS tenant.

## M2S Migration Journey — 3 Phases / 9 Steps

**Plan:** 1. Discover | 2. Strategize | 3. Design

**Upgrade:** 4. Prepare | **5. Execute** | 6. Integrate

**Run:** 7. Enable | 8. Expand | 9. Optimize

---

## Table of Contents

---

- [1. Configuration Migration Strategy](#)
  - [2. Configuration Wave Deployment](#)
  - [3. Manual Configuration Items](#)
  - [4. OneAgent Migration](#)
  - [5. Migration Wave Execution](#)
  - [6. SaaS Upgrade Assistant Deploy Results](#)
  - [7. Step Completion Checklist](#)
-

# Prerequisites

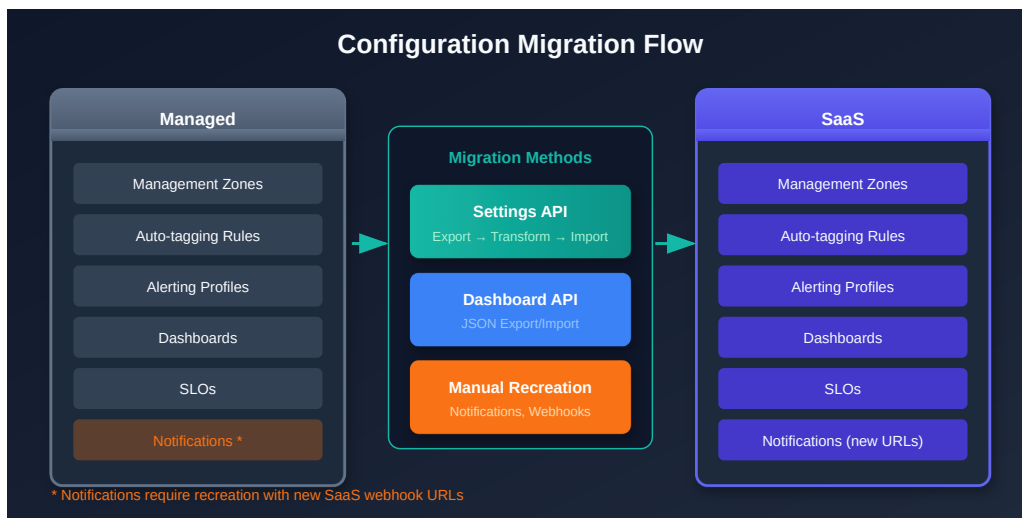
| Requirement                  | Details                                                                                                                               |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Steps 1-4 Complete</b>    | Discovery, strategy, design, and preparation all finished                                                                             |
| <b>SaaS Tenant</b>           | Provisioned with Grail enabled and SaaS Upgrade Assistant installed                                                                   |
| <b>ActiveGates Deployed</b>  | SaaS-connected ActiveGates operational in all network zones (Step 4)                                                                  |
| <b>Network Connectivity</b>  | Firewall rules in place for OneAgent-to-SaaS or OneAgent-to-ActiveGate communication                                                  |
| <b>Configuration Archive</b> | Exported from Managed Cluster Management Console                                                                                      |
| <b>API Tokens</b>            | SaaS tokens with <code>settings.write</code> , <code>entities.read</code> , <code>logs.read</code> , <code>metrics.read</code> scopes |
| <b>Change Windows</b>        | Approved change windows for non-production and production waves                                                                       |

## Order of Operations — You Are Here

This notebook covers operations **4-6** of the 11-step Order of Operations (see **M2S-02: Step 2 — Strategize**) defined in Step 2:

| # | Operation                                       | Status                                                                                                |
|---|-------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 1 | Assess — Inventory current environment          | Step 4: Prepare  |
| 2 | Provision — SaaS tenant and access (SSO)        | Step 4: Prepare  |
| 3 | Install — New ActiveGates in parallel           | Step 4: Prepare  |
| 4 | <b>Migrate — Configuration and integrations</b> | <b>This notebook</b>                                                                                  |
| 5 | <b>Rebuild — Dashboards, zones, alerts</b>      | <b>This notebook</b>                                                                                  |
| 6 | <b>Redirect — OneAgents to SaaS</b>             | <b>This notebook</b>                                                                                  |
| 7 | Reconnect — Integrations and extensions         | Step 6: Integrate                                                                                     |
| 8 | Migrate — Remaining config and integrations     | Step 6: Integrate                                                                                     |

| #  | Operation                            | Status           |
|----|--------------------------------------|------------------|
| 9  | Validate — Data flow and performance | Step 9: Optimize |
| 10 | Cutover — Full switch to SaaS        | Step 9: Optimize |
| 11 | Decommission — Managed environment   | Step 9: Optimize |



## 1. Configuration Migration Strategy

Configurations must be migrated **before** redirecting OneAgents. When agents start reporting to SaaS, they need to find the correct service detection rules, tagging rules, and monitoring settings already in place. Without this, you lose visibility during the transition.

### Migration Order Principle

Configurations FIRST → OneAgents SECOND → Validation AFTER EACH

### SaaS Upgrade Assistant Workflow

1. **Export** — Download the configuration archive from Managed Cluster Management Console
2. **Upload** — Import the archive into the SaaS Upgrade Assistant on your target tenant
3. **Review** — The assistant groups configurations by type and flags incompatible items

4. **Edit** — Fix failed or incompatible configurations individually or in bulk
5. **Deploy** — Use Smart Selective Import to push configurations in dependency-ordered waves
6. **Track** — Download the deploy result CSV after each wave

## Smart Selective Import

The SaaS Upgrade Assistant supports **Smart Selective Import**, which allows you to deploy configurations incrementally rather than all at once. This is critical for a controlled migration:

| Feature                | Benefit                                              |
|------------------------|------------------------------------------------------|
| Dependency awareness   | Automatically identifies prerequisite configurations |
| Incremental deployment | Deploy one configuration type at a time              |
| Preview before deploy  | Review every change before it takes effect           |
| Conflict detection     | Flags configurations that already exist in SaaS      |
| Rollback visibility    | CSV reports show exactly what was deployed           |

**Important:** Always preview changes before deploying. The SaaS Upgrade Assistant shows a diff of what will be created or modified in your SaaS environment.

## 2. Configuration Wave Deployment

Deploy configurations in four waves, ordered by dependency. Each wave builds on the previous one.

### Wave Deployment Order

| Wave   | Configurations                                    | Rationale                                                               |
|--------|---------------------------------------------------|-------------------------------------------------------------------------|
| Wave 1 | Management zones, auto-tagging rules, host groups | Foundation — everything else depends on these for scoping and targeting |

| Wave          | Configurations                                                        | Rationale                                                                                  |
|---------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Wave 2</b> | Service detection rules, request attributes, deep monitoring settings | Instrumentation — must be in place before agents report so services are detected correctly |
| <b>Wave 3</b> | Alerting profiles, anomaly detection, SLOs, maintenance windows       | Monitoring — needs entities present to validate thresholds and baselines                   |
| <b>Wave 4</b> | Dashboards, calculated metrics, remaining configurations              | Visualization — needs data flowing to populate charts and tiles                            |

## Wave 1: Foundation

Deploy first because all other configurations reference management zones and tags:

| Configuration Type | Why First                                                        |
|--------------------|------------------------------------------------------------------|
| Management zones   | Scope alerting, dashboards, and access                           |
| Auto-tagging rules | Tag entities as they appear — must be ready before agents report |
| Host groups        | Group hosts for configuration targeting                          |

## Wave 2: Instrumentation

Deploy before OneAgents migrate so services are detected with the correct naming and grouping:

| Configuration Type       | Why Second                                                        |
|--------------------------|-------------------------------------------------------------------|
| Service detection rules  | Custom service naming and merging must match Managed behavior     |
| Request attributes       | Capture rules for request metadata used in filtering and analysis |
| Deep monitoring settings | Code-level visibility settings (exception rules, method sensors)  |
| Data privacy settings    | Masking rules must be active before data flows                    |

## Wave 3: Monitoring

Deploy after at least some entities exist in SaaS so the system can validate references:

| Configuration Type  | Why Third                                                          |
|---------------------|--------------------------------------------------------------------|
| Alerting profiles   | Filter and route alerts to the correct channels                    |
| Anomaly detection   | Custom thresholds and sensitivity settings                         |
| SLOs                | Service Level Objectives referencing services and management zones |
| Maintenance windows | Prevent false alerts during migration activity                     |

## Wave 4: Visualization

Deploy last because dashboards and calculated metrics need data to be meaningful:

| Configuration Type       | Why Last                                               |
|--------------------------|--------------------------------------------------------|
| Classic dashboards       | Need entities and data to populate tiles               |
| Calculated metrics       | Derive from incoming data — no value without data flow |
| Remaining configurations | Application detection, RUM settings, etc.              |

**After each wave:** Download the deploy result CSV, review failures, fix or document exclusions with justification, then proceed to the next wave.

## Entity-Level Configuration Timing

**Important:** Some entity-level configurations can ONLY be migrated AFTER OneAgents are redirected, because they depend on entities that do not yet exist in the SaaS tenant. Examples: custom settings associated with a specific host or service entity.

| Config Type                                                    | When to Migrate          | Why                     |
|----------------------------------------------------------------|--------------------------|-------------------------|
| <b>Environment-level</b><br>(management zones, tags, alerting) | BEFORE OneAgent redirect | Independent of entities |

| Config Type                                                      | When to Migrate                        | Why                                                                                                                                                                                     |
|------------------------------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Service detection, request attributes                            | BEFORE OneAgent redirect               | Must be in place when services appear                                                                                                                                                   |
| Entity-level custom settings                                     | AFTER OneAgent redirect                | Entity must exist in SaaS first                                                                                                                                                         |
| Deep monitoring, container rules                                 | BEFORE OneAgent redirect               | Needed for correct instrumentation                                                                                                                                                      |
| Grail bucket routing & retention<br>(OpenPipeline Storage stage) | BEFORE high-volume log-source redirect | Records cannot be moved between buckets after ingest; a bucket's name and class (live/historic) are fixed at creation, while retention and the query-billing model can be changed later |

Also note:

- **Process restarts are needed for full-stack mode** — for services to appear in the new SaaS tenant
- **Process restarts are also needed for infra-only mode** — for Application Security vulnerability detection and JMX metrics
- **Minimize delta** with recent configuration changes — this is why the configuration freeze (Step 4) is critical

**Log-heavy migration — decide bucket strategy before logs flow.** Grail bucket layout, retention, and query-billing model are among the most consequential decisions in a log-driven migration, and most of it is **irreversible after first ingest**: records cannot be moved between buckets, and a bucket's name and class (live/historic) are fixed at creation, while retention and the query-billing model can be changed later. Route logs to the right buckets via the OpenPipeline **Storage** stage on the SaaS tenant *before* redirecting high-volume log sources — otherwise everything lands in `default_logs` and cannot be re-bucketed later. See **ORGNZ-03: Bucket Strategy and Design** and **OPLOGS-04: Buckets & Data Governance** for the full bucket/retention design, and confirm bucket routing is in place as part of Step 4 (Prepare).

### 3. Manual Configuration Items

The SaaS Upgrade Assistant handles most configurations, but some items require manual recreation due to security constraints or architectural differences.

#### Items Requiring Manual Recreation

| Item                               | Action                                                                          | Priority                                      |
|------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------|
| <b>Credentials Vault</b>           | Re-enter all credentials in SaaS Credentials Vault — secrets cannot be exported | Before Wave 3 (integrations need credentials) |
| <b>API tokens</b>                  | Generate new tokens with minimal required scopes                                | Before OneAgent migration                     |
| <b>Webhook endpoints</b>           | Create new notification integrations with SaaS-appropriate URLs                 | Before Wave 3 (alerting needs endpoints)      |
| <b>Synthetic private locations</b> | Deploy new Synthetic ActiveGates in SaaS, recreate locations                    | After ActiveGate deployment                   |
| <b>Network zones</b>               | Already created in Step 4 (Prepare) — verify they match Managed topology        | Verify before OneAgent migration              |
| <b>Custom extensions (1.0)</b>     | Rebuild as Extensions 2.0 — EF1 is deprecated                                   | Plan separately (may be post-migration)       |
| <b>Plugin-based integrations</b>   | Migrate to Extensions 2.0 or ActiveGate extensions                              | Plan separately                               |

#### Credentials Vault Checklist

For each credential in Managed, recreate in SaaS:

| Credential Type          | Examples                         | Notes                         |
|--------------------------|----------------------------------|-------------------------------|
| <b>Username/password</b> | Database monitoring, SNMP        | Must be re-entered manually   |
| <b>Token/key</b>         | Third-party integration API keys | Generate new keys if possible |
| <b>Certificate</b>       | mTLS, client certificates        | Upload new certificates       |

## API Token Strategy

Create purpose-specific tokens in SaaS with minimal scopes:

| Token Purpose                | Required Scopes                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------|
| OneAgent installation        | InstallerDownload                                                                                          |
| Configuration API            | settings.read , settings.write                                                                             |
| Entity queries               | entities.read                                                                                              |
| Metrics/logs queries         | metrics.read , logs.read                                                                                   |
| AutomationEngine (Workflows) | OAuth client scopes automation:workflows:read , automation:workflows:write (not a classic API-token scope) |

**Important:** Never reuse Managed API tokens. Generate fresh tokens in SaaS with the principle of least privilege.



## 4. OneAgent Migration

Once configurations are deployed, redirect OneAgents from Managed to SaaS. There are three migration methods — choose based on your requirements.

## Migration Methods

| Method                         | Command                                                                  | Use When                                                                    |
|--------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Reconfigure (preferred)</b> | <code>oneagentctl --set-server +<br/>--set-tenant-token + restart</code> | Standard migration — preserves host identity, minimal downtime              |
| <b>Parallel install</b>        | Install SaaS agent alongside Managed                                     | Zero-gap requirements — both agents report simultaneously during transition |
| <b>Reinstall</b>               | Uninstall Managed agent, install SaaS agent                              | Clean start needed — resets all agent state                                 |

### Reconfigure Method (Recommended)

The reconfigure method is preferred because it preserves host identity and requires only a brief restart:

```
# 1. Set new server endpoint
oneagentctl --set-server="https://{tenant-id}.live.dynatrace.com:443/communication"

# 2. Set new tenant token (from SaaS: Settings > Deployment > Installer download)
oneagentctl --set-tenant-token="{tenant-token}"

# 3. Set network zone (if applicable — must match zones created in Step 4)
oneagentctl --set-network-zone=&lt;zone-name&gt;

# 4. Restart OneAgent
# Linux:
systemctl restart oneagent
# Windows:
Restart-Service -Name "Dynatrace OneAgent"
```

### If Using ActiveGate Routing

When OneAgents connect through ActiveGates rather than directly to SaaS:

```
# Point to ActiveGate instead of SaaS directly
oneagentctl --set-server="https://{activegate-host}:9999/communication"
oneagentctl --set-tenant-token="{tenant-token}"
oneagentctl --set-network-zone=&lt;zone-name&gt;
systemctl restart oneagent
```

## Critical: Restart Application Processes

**Warning:** Application processes **MUST** be restarted after OneAgent migration. Without a restart, you get host-level metrics but **NO distributed tracing, NO service detection, and NO code-level visibility**.

| What Works Without Restart     | What Requires Restart           |
|--------------------------------|---------------------------------|
| Host CPU, memory, disk metrics | Distributed tracing (PurePaths) |
| Network metrics                | Service detection and naming    |
| Process-level metrics          | Code-level exception analysis   |
| Log monitoring                 | Request attributes capture      |
|                                | Method-level hotspots           |

Plan application restarts as part of your migration wave — coordinate with application teams for scheduled restart windows.

## Parallel Install Method

For environments requiring zero monitoring gaps:

1. Install the SaaS OneAgent alongside the existing Managed agent
2. Validate data flows to SaaS for the configured period
3. Uninstall the Managed agent once SaaS is confirmed
4. Restart application processes

**Note:** Running two agents simultaneously doubles the CPU and memory overhead on the host. Use this method only when monitoring continuity is a hard requirement.

**Tip — OneAgent Attribute Enrichment (1.333+):** During agent redirect, consider adding primary tags and fields at the same time using `oneagentctl --set-host-tag`. This enriches all telemetry at the source with `primary_tags.environment`, `dt.security_context`, `dt.cost.costcenter`, etc. — eliminating the need for some server-side auto-tagging rules. See [docs](#).

## Redirecting Containerized and Serverless Workloads

The `oneagentctl` reconfigure method above applies to **host-based OneAgents** (VMs and bare metal). Containerized and serverless workloads have no long-lived host to reconfigure, so the redirect happens through the deployment surface — the operator custom resource, the image build, or the exporter endpoint — not through `oneagentctl`.

| Surface                                                 | Redirect mechanism                                                                                                                                                                                        | Restart / redeploy                                                                         |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Host VM</b> (GCE / EC2 / on-prem)                    | <code>oneagentctl --set-server + --set-tenant-token (+ --set-network-zone)</code> , as above                                                                                                              | Restart the OneAgent service <b>and</b> restart app processes to re-inject deep monitoring |
| <b>Kubernetes / GKE</b> (Dynatrace Operator / DynaKube) | <code>spec.apiUrl</code> is <b>immutable</b> (Operator 1.3.0+): delete the DynaKube custom resource and its token secret, recreate both against the new <code>apiUrl</code> , then roll the workloads     | Rolling pod restart adopts the new tenant; no node restart                                 |
| <b>Cloud Run — Java / Node.js</b> (OneAgent)            | OneAgent tenant credentials are baked in as <b>build-time image arguments</b> ( <code>DT_API_URL</code> , <code>DT_API_TOKEN</code> ): rebuild the image against the new tenant and deploy a new revision | The new revision <i>is</i> both the redirect and the re-injection                          |
| <b>Cloud Run — other runtimes / OTLP apps</b>           | Repoint the OTLP exporter endpoint and API token, then deploy a new revision                                                                                                                              | New revision                                                                               |

**Key difference from VMs:** On containerized and serverless surfaces the redirect and the deep-code re-injection are the **same action** — the rolled pods or the new revision. There is no separate "now also restart your applications" step the way there is on a VM, because the workload is recreated from scratch on every deploy.

**Cloud Run runtime caveat:** OneAgent monitoring of Cloud Run managed is currently limited to **Java and Node.js**; other runtimes (Go, Python, .NET) report via **OTLP** rather than OneAgent — verify current runtime support before planning the redirect. Cloud Run instances appear on the **Hosts** page, and first-generation execution environments do not expose CPU/memory metrics.

## 5. Migration Wave Execution

Execute the OneAgent migration in waves, starting with non-production and progressing to production.

**Capacity pre-warm for large waves:** If a single wave will connect more than ~1,000 hosts to the SaaS tenant within a few hours, notify your Dynatrace account team in advance so the tenant can be pre-scaled for the ingest surge. Pacing large fleets this way avoids throttling and delayed host check-in.

### Wave Execution Order

| Wave   | Environment          | Hosts          | Purpose                             |
|--------|----------------------|----------------|-------------------------------------|
| Wave A | Development/Test     | 5-10 hosts     | Validate process, identify issues   |
| Wave B | Non-production (all) | All non-prod   | Confirm at scale, validate services |
| Wave C | Production (pilot)   | 10-20% of prod | Validate production behavior        |
| Wave D | Production (full)    | Remaining prod | Complete migration                  |

### Per-Wave Procedure

For each wave, follow this sequence:

1. **Pre-check** — Verify SaaS ActiveGates are healthy and configurations are deployed
2. **Reconfigure agents** — Run `oneagentctl` commands (manually or via automation)
3. **Restart OneAgent** — Restart the agent service on each host
4. **Verify host check-in** — Confirm hosts appear in SaaS within 5 minutes
5. **Restart applications** — Coordinate with application teams for process restarts
6. **Validate data flow** — Run the DQL queries below to confirm metrics, traces, and logs

7. **Document results** — Record successes, failures, and lessons learned

8. **Apply lessons** — Adjust procedures for subsequent waves

## Automation for Bulk Migration

For large environments, use automation tools to execute the reconfigure commands across many hosts:

```
# Ansible example – reconfigure OneAgent on all hosts in a group
ansible-playbook -i inventory/nonprod migrate_oneagent.yml \
  --extra-vars "tenant_url=https://{tenant-id}.live.dynatrace.com:443/communication \
               tenant_token={token} \
               network_zone=datacenter-east"
```

## Post-Wave Validation

After each wave, run the following DQL queries against your SaaS tenant to verify data flow.

## Verify Hosts Reporting to SaaS

```
// Count hosts reporting to SaaS – compare against expected count from
migration wave
fetch dt.entity.host
| summarize hostCount = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize hostCount = count()
```

## Check for Metric Data Gaps

```
// Verify CPU metrics are flowing – hosts with data in the last hour
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd hasData = arrayAvg(avgCpu)
| filter isNotNull(hasData)
| summarize hostsWithData = count()
```

## Verify Services Discovered

```
// Count discovered services – should grow as applications restart
fetch dt.entity.service
| summarize serviceCount = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize serviceCount = count()
```

## Check Distributed Traces Flowing

```
// Verify spans are arriving – traces require application process restart
fetch spans, from:-1h
| summarize spanCount = count(), by:{time_bucket = bin(start_time, 5m)}
| sort time_bucket desc
| limit 12
```

## Verify Log Ingestion

```
// Confirm log data is flowing from migrated hosts
fetch logs, from:-1h
| summarize logCount = count(), by:{time_bucket = bin(timestamp, 5m)}
| sort time_bucket desc
| limit 12
```

## Compare Expected vs Actual Host Count

After each wave, compare the expected host count (from your migration plan) against actual hosts reporting:

```
// Hosts grouped by OS type – verify all expected host types are present
fetch dt.entity.host
| summarize hostCount = count(), by:{osType}
| sort hostCount desc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize hostCount = count(), by:{osType}
// | sort hostCount desc
```

**Tip:** If span counts are zero but host metrics are flowing, application processes have not been restarted yet. Coordinate restarts before declaring the wave complete.

## 6. SaaS Upgrade Assistant Deploy Results

After each configuration wave deployment, download and review the deploy result CSV from the SaaS Upgrade Assistant.

### Result Categories

| Status   | Meaning                                                     | Action                               |
|----------|-------------------------------------------------------------|--------------------------------------|
| Deployed | Configuration successfully created or updated in SaaS       | No action needed                     |
| Skipped  | Configuration already exists in SaaS (no overwrite)         | Verify existing config is correct    |
| Failed   | Configuration could not be deployed (incompatible or error) | Fix and retry, or document exclusion |
| Excluded | Manually excluded from deployment                           | Document justification               |

### Handling Failed Configurations

For each failed configuration:

1. **Identify the failure reason** — The CSV includes error details

2. **Determine if fixable** — Can the configuration be edited in the SaaS Upgrade Assistant?
3. **Fix and retry** — Edit the configuration and redeploy
4. **Or exclude with justification** — If the configuration is no longer needed or will be rebuilt differently in SaaS

## Common Failure Reasons

| Failure                        | Cause                                       | Resolution                                                         |
|--------------------------------|---------------------------------------------|--------------------------------------------------------------------|
| Entity reference not found     | Entity ID changed between Managed and SaaS  | Update entity reference or let it auto-resolve after agents report |
| Unsupported configuration type | Legacy feature not available in SaaS        | Rebuild using SaaS-native equivalent                               |
| Duplicate name conflict        | Configuration with same name already exists | Rename or merge with existing                                      |
| Permission denied              | Missing IAM policy                          | Add <code>upgrade-assistant:environments:write</code> policy       |

## Archiving Deploy Results

Archive all deploy result CSVs for compliance and audit purposes:

```
deploy-results/  
  wave-1-foundation-YYYY-MM-DD.csv  
  wave-2-instrumentation-YYYY-MM-DD.csv  
  wave-3-monitoring-YYYY-MM-DD.csv  
  wave-4-visualization-YYYY-MM-DD.csv
```

**Best Practice:** Store deploy result CSVs in your migration project repository alongside your migration runbook. This creates an auditable record of every configuration change.

## 7. Step Completion Checklist

Before proceeding to Step 6, confirm that you have completed each item:

| Checkpoint                                                                | Status |
|---------------------------------------------------------------------------|--------|
| Wave 1 deployed: management zones, auto-tagging rules, host groups        | [ ]    |
| Wave 2 deployed: service detection, request attributes, deep monitoring   | [ ]    |
| Wave 3 deployed: alerting profiles, anomaly detection, SLOs               | [ ]    |
| Wave 4 deployed: dashboards, calculated metrics, remaining configs        | [ ]    |
| Manual items recreated (credentials vault, API tokens, webhook endpoints) | [ ]    |
| All deploy result CSVs downloaded and archived                            | [ ]    |
| Non-production OneAgents migrated and validated                           | [ ]    |
| Production OneAgents migrated and validated                               | [ ]    |
| Application processes restarted (all waves)                               | [ ]    |
| Data flow confirmed: host metrics flowing                                 | [ ]    |
| Data flow confirmed: distributed traces flowing                           | [ ]    |
| Data flow confirmed: log ingestion active                                 | [ ]    |

## Next Step

**M2S-06: Step 6 — Integrate** — Reconnect all integrations and extensions. Migrate notification channels, rebuild Extensions 2.0, configure cloud integrations, and validate end-to-end alerting workflows.

## Additional Resources

- [SaaS Upgrade Assistant Documentation](#)
- [OneAgent Configuration via Command Line](#)
- [Migrate Dynatrace Operator to a New Environment](#)

- [Monitor Google Cloud Run \(Managed\)](#)
  - [Network Zones Configuration](#)
  - [Extensions 2.0 Framework](#)
- 

## Summary

---

In Step 5, you:

- Deployed configurations to SaaS in four dependency-ordered waves using the SaaS Upgrade Assistant
- Recreated manual configuration items (credentials vault, API tokens, webhook endpoints)
- Migrated OneAgents from Managed to SaaS using the reconfigure method
- Restarted application processes for full distributed tracing and service detection
- Validated data flow (metrics, traces, logs) after each migration wave
- Archived all deploy result CSVs for compliance

**Key Takeaway:** Migrate configurations before agents, validate after every wave, and always restart application processes. Without application restarts, you see host metrics but lose distributed tracing and service detection.

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*